

ImageDiskCache

A lightweight utility library for caching network images on disk in Flutter. `ImageDiskCache` downloads an image from a URL, stores it locally, and reuses the cached file on subsequent requests to improve performance and reduce network usage.

✨ Features

- 📦 Automatic disk caching of images
 - 🚀 Faster image loading after first download
 - 🌐 Works with any HTTP image URL
 - 📄 Simple, static API (no instantiation needed)
 - 📁 Uses application documents directory for persistence
-

🐘 Library Declaration

```
library image_disk_cache;
```

This allows the utility to be imported as a standalone library.

🦍 Dependencies

```
import 'dart:io';  
import 'package:http/http.dart' as http;  
import 'package:path_provider/path_provider.dart';
```

- **dart:io** → File system access
 - **http** → Downloading images from the network
 - **path_provider** → Locating a persistent storage directory
-

🍁 Public API

`getImage`

```
static Future<File> getImage(String imageUrl)
```

Description

Returns a locally cached image file if it exists. Otherwise, downloads the image from the given URL, saves it to disk, and then returns the file.

Parameters

Name	Type	Description
<code>imageUrl</code>	<code>String</code>	Network image URL

Returns

- `Future<File>` → The image file stored on disk

Flow

1. Generate a local file path based on the URL hash
2. Check if the file already exists
3. If yes → return cached file
4. If no → download and save the image



Internal Methods

`_getLocalFile`

```
static Future<File> _getLocalFile(String imageUrl)
```

Generates a deterministic local file path using the image URL's hash code.

Why hashCode?

Ensures unique file names per URL while keeping paths short and filesystem-safe.

`_downloadAndSave`

```
static Future<File> _downloadAndSave(String imageUrl, File file)
```

Downloads the image from the network and writes the bytes directly to disk.

Usage Example

```
final file = await ImageDiskCache.getImage(  
  'https://example.com/image.jpg',  
);  
  
Image.file(file);
```

This will: - Download the image the first time - Load instantly from disk on subsequent calls

Notes & Limitations

-  No cache expiration or size limit
 -  No error handling for failed HTTP requests
 -  Images are always saved with `.jpg` extension
-

Suggested Enhancements

- Add cache expiration / max size
 - Support original file extensions
 - Handle HTTP errors & timeouts
 - Add memory cache layer
 - Allow custom cache directory
-

Full Source Code

```
library image_disk_cache;  
  
import 'dart:io';  
import 'package:http/http.dart' as http;  
import 'package:path_provider/path_provider.dart';  
  
class ImageDiskCache {  
  static Future<File> getImage(String imageUrl) async {  
    final File file = await _getLocalFile(imageUrl);  
  
    if (await file.exists()) {  
      return file;  
    }  
  }  
}
```

```

        return await _downloadAndSave(imageUrl, file);
    }

    static Future<File> _getLocalFile(String imageUrl) async {
        final directory = await getApplicationDocumentsDirectory();
        final fileName = imageUrl.hashCode.toString();
        final filePath = '${directory.path}/${fileName}.jpg';

        return File(filePath);
    }

    static Future<File> _downloadAndSave(
        String imageUrl,
        File file,
    ) async {
        final response = await http.get(Uri.parse(imageUrl));
        await file.writeAsBytes(response.bodyBytes);
        return file;
    }
}

```



When to Use This

- Small to medium apps
- Simple image caching needs
- When you want full control without external caching packages

If you want, I can: - Add **error handling + retries** - Convert this into a **Flutter package** - Add **cache eviction logic** - Create a **drop-in Image widget** wrapper

Just say the word 